

Reporte Técnico: Configuración de Logging con Serilog y Seq

Materia: Seguridad en el Desarrollo de Aplicaciones Proyecto: VulnerableApp

1. Objetivo

Configurar Serilog como plataforma centralizada de logging para el proyecto VulnerableApp y validar la generación de registros en consola, archivo de texto y mediante el visualizador de eventos Seq.

2. Actividades Realizadas

Para lograr el objetivo, se llevaron a cabo los siguientes pasos:

Instalación de paquetes NuGet: Se instalaron los paquetes base de Serilog (Serilog.AspNetCore) y los correspondientes para cada sumidero (Sink): Serilog.Sinks.Console, Serilog.Sinks.File y Serilog.Sinks.Seq. Adicionalmente, se instaló Serilog.Enrichers.Environment.

Configuración en Program.cs: Se inicializó LoggerConfiguration en la canalización principal (pipeline) de .NET para redirigir los logs a los 3 Sinks.

Carpeta Logs: Se configuró Serilog para que genere y almacene dinámicamente los registros dentro del directorio /Logs/log-.txt.

Instalación de Seq: Se configuró e inicializó Seq mediante un contenedor de Docker local, mapeando el puerto 5341 para validar la conectividad y recepción de eventos estructurados.

3. Resolución de Retos

A. Función de cada Sink (Sumidero)

Console Sink (WriteTo.Console()): Intercepta los logs y los imprime directamente en la terminal donde se ejecuta la aplicación. Es útil durante la etapa de desarrollo para tener retroalimentación visual en tiempo real.

File Sink (WriteTo.File("Logs/log-.txt")): Escribe permanentemente los registros en un archivo de texto en disco. En la configuración se especificó RollingInterval.Day, por lo cual se crea un archivo nuevo cada día. Esto evita la saturación de espacio y asegura un almacenamiento histórico para auditorías.

Seq Sink (WriteTo.Seq("http://localhost:5341")): Envía los logs de forma asíncrona y estructurada (JSON) a un servidor Seq. Su función es centralizar los logs y proveer una interfaz web para buscar, filtrar y analizar rápidamente errores en sistemas complejos.

B. Nivel mínimo de logging desde appsettings.json

Para evitar tener el nivel de log codificado "en duro" (MinimumLevel.Information()), se modificó el código usando .ReadFrom.Configuration(builder.Configuration). Esto permitió que la aplicación lea directamente el bloque "Serilog": { "MinimumLevel": { "Default": "Information" } } definido en el archivo appsettings.json. De esta manera, el nivel de detalle de los logs puede alterarse en entornos de producción simplemente modificando el archivo JSON, sin necesidad de recompilar el código.

C. Enriquecedor Adicional y su Justificación

Se implementó el enriquecedor `.Enrich.WithMachineName()` (proporcionado por el paquete `Serilog.Enrichers.Environment`).

Justificación de uso: En arquitecturas de software modernas (como microservicios o contenedores balanceados en la nube), una misma aplicación suele ejecutarse en múltiples servidores o instancias al mismo tiempo. Al agregar automáticamente el nombre de la máquina a cada registro, podemos identificar de forma inmediata qué servidor físico o contenedor virtual específico está experimentando problemas, reduciendo el tiempo de diagnóstico y depuración en Seq.

4. Pruebas y Evidencias

4.1. Configuración de Program.cs

Se configuró correctamente el inicio del Logger.

```
using Microsoft.EntityFrameworkCore;
using Serilog;
using VulnerableApp.Data;

var builder = WebApplication.CreateBuilder(args);

// Configurar Serilog
Log.Logger = new LoggerConfiguration()
    .ReadFrom.Configuration(builder.Configuration) // Lee el nivel mínimo y otros
    .ajustes desde appsettings.json
    .WriteTo.Console()
    .WriteTo.File("Logs/log-.txt", rollingInterval: RollingInterval.Day)
    .WriteTo.Seq("http://localhost:5341")
    .Enrich.FromLogContext()
    .Enrich.WithMachineName() // Enriquecedor adicional
    .CreateLogger();

builder.Host.UseSerilog();

// Add services to the container.
builder.Services.AddControllersWithViews();

// Registrar el DbContext usando la cadena de conexión de appsettings.json
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnec
tion")));

builder.Services.AddSession();
```

```

var app = builder.Build();

// Agregar Serilog Request Logging (opcional, pero muy útil para registrar
// peticiones HTTP)
app.UseSerilogRequestLogging();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();
app.UseSession();
app.UseAuthorization();
app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();

// --- HACK PARA IMPRIMIR LA TABLA EN CONSOLA ---
using (var scope = app.Services.CreateScope())
{
    var context =
scope.ServiceProvider.GetRequiredService<VulnerableApp.Data.AppDbContext>();

    Console.WriteLine("\n=====");
};

Console.WriteLine(" EVIDENCIA 3: DATOS EN TEXTO PLANO (VulnerableDb)");
Console.WriteLine("=====");

// Agregamos await y cambiamos a ToListAsync()
foreach (var u in await context.Users.ToListAsync())
{
    Console.WriteLine($" ID: {u.Id} | Usuario: {u.Username} | PasswordHash:
{u.PasswordHash}");
}

```

```

        Console.WriteLine("=====\\n"
    );
}

// Agregamos await y cambiamos a RunAsync()
await app.RunAsync();

```

4.2. Generación de eventos en Consola

Al ejecutar la aplicación, se validó la impresión de solicitudes HTTP (registros INF) usando colores en la consola.

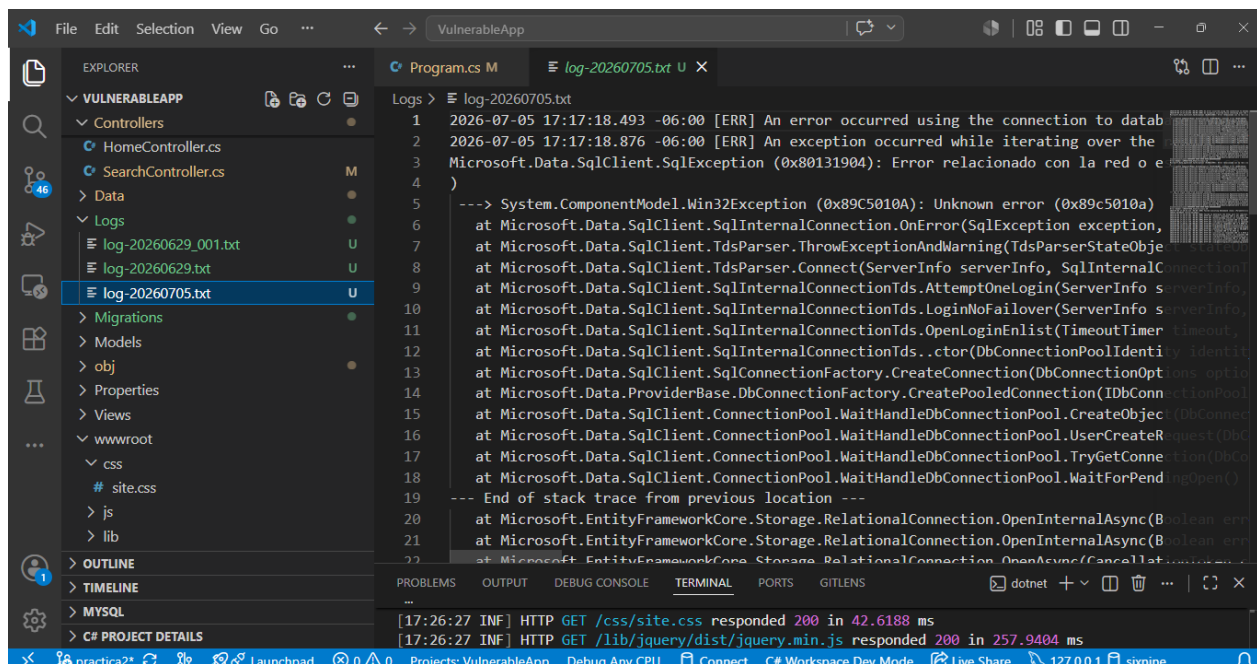
```

[17:26:26 WRN] Failed to determine the https port for redirect.
[17:26:27 INF] HTTP GET /Comment responded 200 in 871.9996 ms
[17:26:27 INF] HTTP GET /VulnerableApp.styles.css responded 200 in 46.9186 ms
[17:26:27 INF] HTTP GET /css/site.css responded 200 in 42.6188 ms
[17:26:27 INF] HTTP GET /lib/jquery/dist/jquery.min.js responded 200 in 257.9404 ms
[17:26:27 INF] HTTP GET /js/site.js responded 200 in 9.9582 ms
[17:26:27 INF] HTTP GET /lib/bootstrap/dist/css/bootstrap.min.css responded 200 in 261.5043 ms
[17:26:27 INF] HTTP GET /lib/bootstrap/dist/js/bootstrap.bundle.min.js responded 200 in 263.4638 ms
[17:26:28 INF] HTTP GET /favicon.ico responded 200 in 38.3440 ms
[17:26:32 INF] HTTP GET / responded 200 in 92.5935 ms
[17:26:32 INF] HTTP GET /css/site.css responded 200 in 3.0723 ms
[17:26:32 INF] HTTP GET /VulnerableApp.styles.css responded 200 in 1.9488 ms
[17:26:32 INF] HTTP GET /lib/bootstrap/dist/css/bootstrap.min.css responded 200 in 68.3594 ms
[17:26:32 INF] HTTP GET /js/site.js responded 200 in 29.3084 ms
[17:26:32 INF] HTTP GET /lib/jquery/dist/jquery.min.js responded 200 in 119.6178 ms
[17:26:32 INF] HTTP GET /lib/bootstrap/dist/js/bootstrap.bundle.min.js responded 200 in 184.0371 ms

```

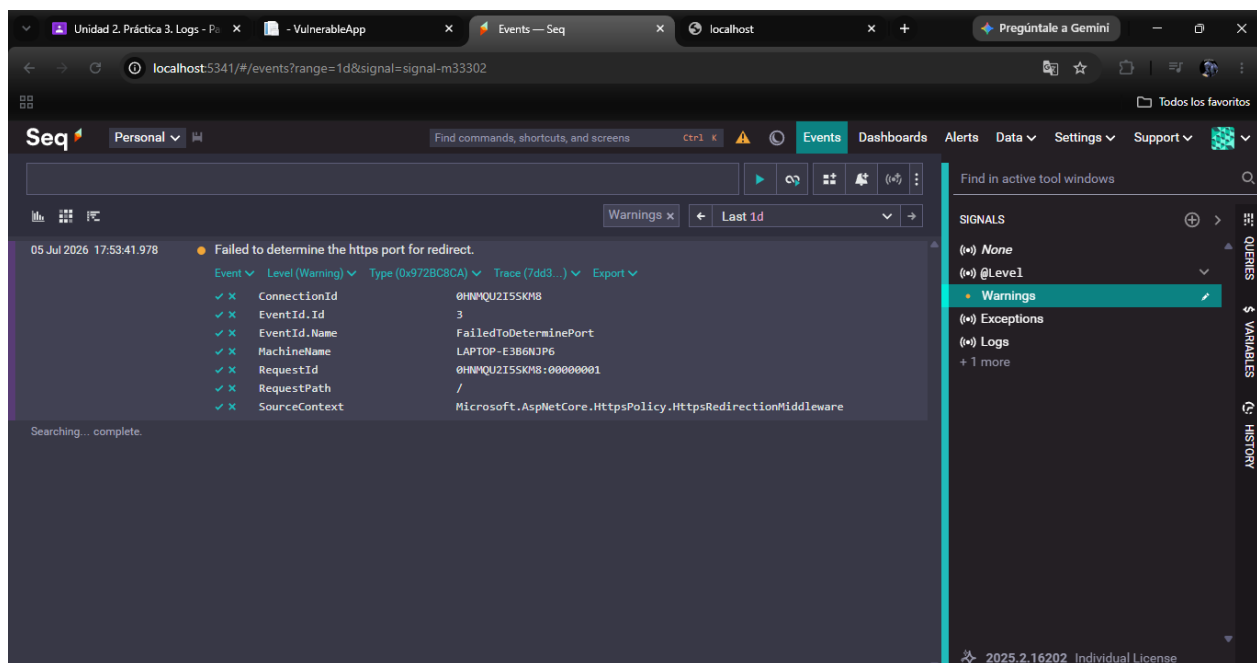
4.3. Creación de archivos de log

Se corroboró la escritura física de registros en disco.



4.4. Eventos centralizados en Seq

Se verificó que los eventos estructurados están llegando a la URL del contenedor Docker (localhost:5341), incluyendo la propiedad MachineName del enriquecedor.



4.5. Prueba de cambio de nivel de logging

Se modificó el valor "Default" de "Information" a "Warning" en el appsettings.json y se reinició la aplicación.

Resultado observado: La aplicación dejó de imprimir las líneas de éxito (rutas HTTP 200). Únicamente se registrarán eventos si ocurre alguna advertencia o excepción crítica, lo cual

comprueba que la configuración mediante JSON funciona correctamente y reduce el ruido en los logs.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS  dotnet + - [] [X] ... | [C] [X]

○ PS D:\Documentos\ing_2doCuatri\Seguridad en desarrollo de aplicaciones\Practica2\VulnerableApp> dotnet
  run
Usando la configuración de inicio de D:\Documentos\ing_2doCuatri\Seguridad en desarrollo de aplicacion
es\Practica2\VulnerableApp\Properties\launchSettings.json...
Compilando...

=====
EVIDENCIA 3: DATOS EN TEXTO PLANO (VulnerableDb)
=====
ID: 1 | Usuario: admin | PasswordHash: $2a$11$wZf6qk90DzD6jpXbs9ljhuplDCeIPd7wGxuvdBIp07HvYccdPVUjq
ID: 2 | Usuario: user1 | PasswordHash: $2a$11$YhH73NAPzWhdxKa8gmBn40qybk8HpkL40FMgR9T0.U7mIzW6Bn1V.
ID: 3 | Usuario: user2 | PasswordHash: $2a$11$oV81ldHlBORG8Fyh/c8Kl.WOhHJ5ys9ipQr2NPvLScq0gdrU4gQV2
=====

[17:57:40 WRN] Failed to determine the https port for redirect.
|
```